

Luego de leer el enunciado completo, se aconseja seguir estos pasos:

1. Describir CONTEXTO, OBJETIVO, ACTORES y FUNCIONALIDADES:

¿Dónde está ubicado mi dominio?

¿Cuál es el objetivo del sistema? ¿Qué espera el cliente que resolvamos?

¿Qué actores participan del sistema? ¿Qué hace cada uno? → CASOS DE USO.

Pensemos al sistema como una “caja negra”: ¿Qué necesita cumplir el sistema? ¿Para quién? Ahora no nos interesa cómo (luego sí, pero no ahora).

Hay que definir el alcance del proyecto, para saber qué está dentro y qué está fuera.

2. Obtener un listado de REQUERIMIENTOS y RESTRICCIONES:

Requerimientos → qué esperan los clientes del comportamiento del sistema.

Restricciones → qué limitaciones se deben considerar.

3. Describir un ESCENARIO:

Describir una secuencia concreta de ejecuciones de funcionalidades que van a llevar a cabo los actores. Para eso, describir el ciclo de vida o la forma de operar de las entidades de las que somos responsables.

Armar el escenario lleva a tener que pensar qué cosas de antemano tengo que tener cargadas o definidas antes de que un actor interactúe, ya que éste hará uso de ellas.

4. Listar las FUNCIONALIDADES:

Serán ejecutadas por los distintos actores, en orden, contra nuestro sistema para recorrer el ciclo de vida de inicio a fin de la entidad principal.

Es importante detectar cuáles funcionalidades son ejecutadas como resultado de algún evento y cuáles funcionalidades son ejecutadas directamente desde un actor.

5. Armar el MODELO:

Debe incluir elementos de gran influencia y omitir los irrelevantes, para el nivel de abstracción buscado (siempre considerando a quién se le va a presentar).

A medida que se va armando el modelo es necesario validar que los requerimientos se cumplan y que se respeten los principios que guían a un buen diseño.

A la larga tal vez sea necesario crear o replantear algunas abstracciones y/o responsabilidades asignadas, dado que al construir el modelo se va comprendiendo mejor el sistema y la misma solución planteada. Muchas veces descubrimos oportunidades para la simplificación, la reutilización o reorganización.

Temas recurrentes:

- **Arquitectura:**
 - o Atributos de Calidad.
 - o Cualidades de Diseño.
 - o Estilos y Patrones Arquitectónicos.
 - o Estrategias de Integración.
 - o Diagrama de Componentes.
 - o Diagrama de Despliegue.
- **Modelo de Dominio:**
 - o Diagrama de Clases.
 - o Patrones de Diseño.
 - o Diagrama de Secuencia.
 - o Diagrama de Estados.
 - o Estrategias de Mapeo Objeto-Relacional (ORM).
- **Persistencia:**
 - o Diagrama Entidad-Relación (DER).
 - o Estrategias de Mapeo Objeto-Relacional (ORM).

Duración: 120' o 150', según el examen.

Si la consigna no aclara algo, aclarar supuestos o decisiones que se tomaron.

Para aprobar, es necesario:

- Completar todos los requerimientos.
- Que los diagramas realizados sean consistentes entre sí y con la arquitectura.
- Justificar todas las decisiones de diseño posibles: por qué se eligió cierto patrón de diseño y no otro, por qué se implementó cierta estrategia ORM y no otra, etcétera.

- A la arquitectura hay que entenderla mirándola de arriba.
Es importante entender qué nodos/componentes participan e interactúan.
- En la comparación de arquitecturas mediante atributos de calidad:
 - o Debemos comparar un componente o una funcionalidad en particular de una arquitectura contra un componente o una funcionalidad de otra arquitectura.
No debemos comparar arquitecturas completas, sino partes.
 - o Debemos aclarar un aspecto o un escenario concreto donde se puedan ver las diferencias de ese atributo de calidad entre tales arquitecturas.
Por ejemplo: “*Analizamos <ATRIBUTO DE CALIDAD> entre las arquitecturas, particularmente en términos de <ASPECTO O ESCENARIO CONCRETO>: La arquitectura A es más <...> que la arquitectura B porque <...>*”.

- Si algo tiene comportamiento, probablemente sea una clase o una interfaz.
Si algo no tiene comportamiento, no será una clase.
- Dos clases distintas ↔ los atributos y métodos son distintos.
Dos instancias de una misma clase ↔ los atributos y métodos son los mismos, pero con valores distintos.
- NO es obligatorio el uso de patrones de diseño.
Si se los usara, debo:
 - o Saber cómo persistirlos en el DER.
 - o Justificar su elección.
 - o Explicar qué otras alternativas tenía y por qué fueron descartadas.
- Sobre las relaciones entre las clases:
 - o Agregar cardinalidades.
 - o Agregar verbos sobre las relaciones (para entenderlas mejor), si es posible
 - o Ante la duda, usar la relación de asociación, sin especificar cardinalidad.
- En las clases, por más que a veces no se pida, es recomendable agregar ciertos atributos:
 - o **fechaHora**, para guardar cuándo se hizo cierta operación → **trazabilidad**.
 - o **usuario**, para guardar quién hizo cierta operación → **auditoría**.

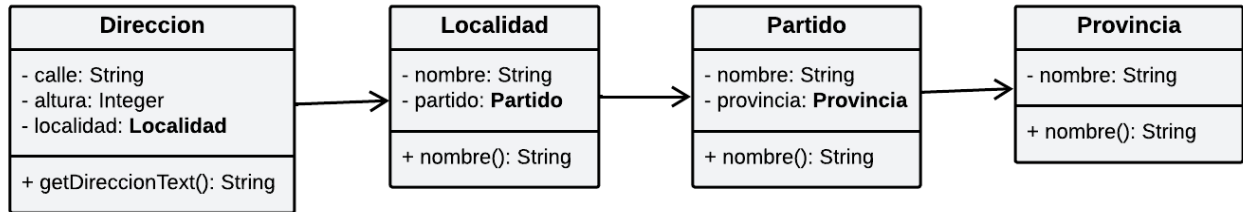
CLASE vs **ENUM**:

- Se analiza el comportamiento:
 - o Si tiene comportamiento, no será un ENUM → será una CLASE.
 - o Si no tiene comportamiento, puede ser cualquiera de las dos.
- Se analiza la extensibilidad (pudiendo agregarse nuevas entidades en *runtime*):
 - o Si debe ser extensible → será una CLASE.
 - o Si no debe ser extensible → será un ENUM.

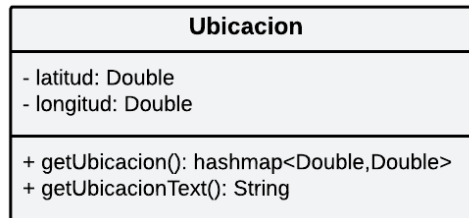
HERENCIA vs **STATE** vs **STRATEGY**:

- **HERENCIA** → de rol estático, no es posible cambiar entre una cosa y otra.
 - o Es algo rígido, no se puede cambiar.
- **STATE** → de rol dinámico, a veces es una cosa y a veces es otra.
 - o Hay una relación entre los *estados*, se “conocen”.
 - o Es posible hacer un diagrama de estados entre los *estados*.
- **STRATEGY** → de rol dinámico, a veces es una cosa y a veces, otra.
 - o NO hay relación entre las *estrategias*, no se “conocen”.
 - o NO es posible hacer un diagrama de estados entre las *estrategias*.

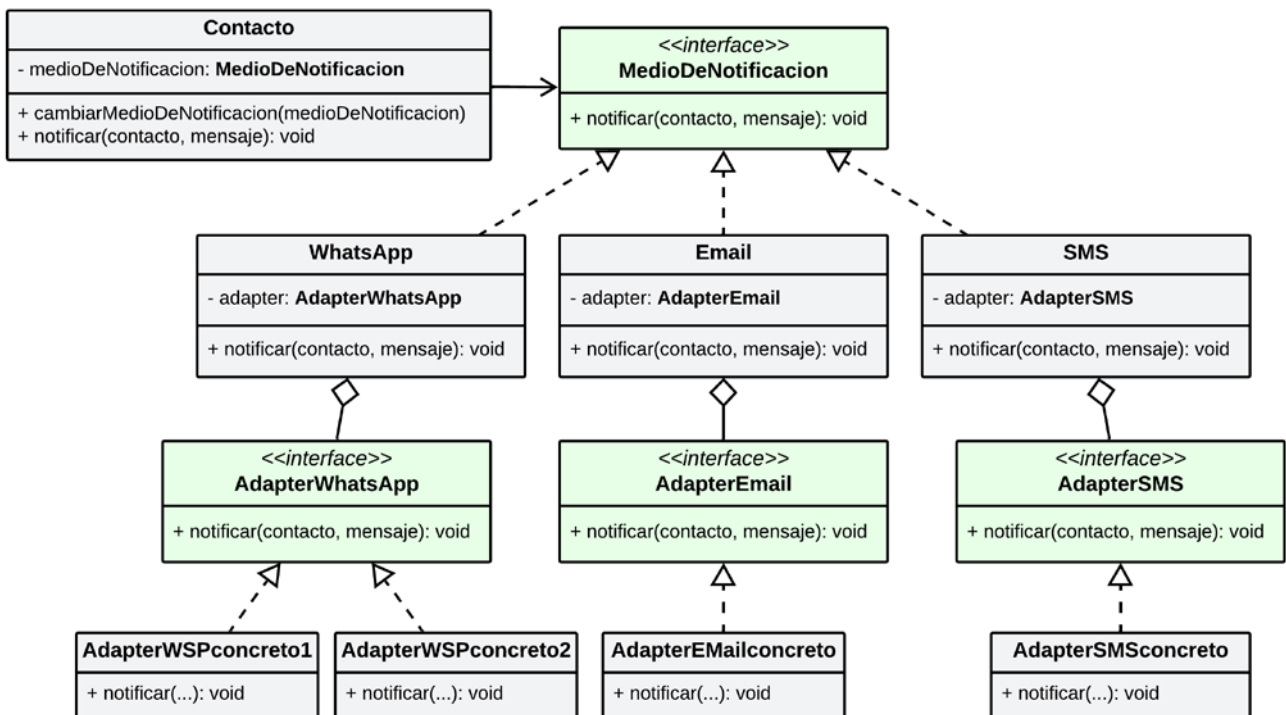
¿Cómo modelar una **dirección**, su **localidad**, **partido** y **provincia**?



¿Cómo modelar una **ubicación**?



¿Cómo modelar los **medios de notificación** (con *STRATEGY* y *ADAPTER*)?



- Debo analizar qué persistir y qué no...

Si hay que persistir algo, analizar si alcanza con persistir sólo un campo dentro de una entidad/tabla o si se requiere una entidad/tabla (algo de mayor importancia).

- En las entidades/tablas:
 - o Deben indicarse los campos claves (PKs, FKs).
 - o Es recomendable indicar los tipos de datos.
 - o Por más que a veces no se pida, es recomendable agregar:
 - Un campo `fechaHora`, para guardar cuándo se hizo cierta operación (datos históricos) y, así, garantizar **trazabilidad**.
 - Un campo `usuario`, para guardar quién hizo cierta operación y, así, garantizar **auditoría**.
- Sobre normalizar o desnormalizar:
 - o Si se busca normalizar, con la 3FN es suficiente.
 - o Si se quiere desnormalizar, justificar por qué se hace (si por consistencia o por performance).

Del DC (diagrama de clases) al DER (diagrama entidad-relación):

- Cada objeto en el **DC** debe poder ser identificado de manera unívoca en el **DER**.
- ¿Qué clases del **DC** deben persistirse en el **DER**?
 - o Si una clase sólo tiene métodos (no tiene atributos), NO se persiste.
 - o Las interfaces del **DC** nunca se mapean al **DER**.
 - o Si se tiene un *ADAPTER*, por lo general NO se persiste (salvo que se pida).
 - o Si se tiene un *STRATEGY*, por lo general SÍ se persiste (aunque depende del contexto y de los requerimientos).

Ejemplo: se modela con un STRATEGY a los medios de notificación que elige el usuario para ser notificado y se quiere que se sepa cómo el usuario eligió que lo notifiquen.

Se persiste la estrategia, pero no necesariamente se persiste una entidad. Si cada estrategia no tiene datos que persistir, entonces sólo se persiste una simple etiqueta (un campo).
- Al elegir la estrategia de mapeo (single table, table per class o table per subclass):
 - o Si es muy evidente cuál usar, justificar la elección.
 - o Si no es muy evidente cuál usar, se aconseja usar single table (por simplicidad).
- Respecto de las relaciones:
 - o Toda **composición** y **agregación** será una relación one-to-many o many-to-many.
 - o Respecto de un atributo de una clase:
 - Si es simple, la relación será one-to-one o many-to-one.
 - Si es una colección, la relación será one-to-many o many-to-many.
- Respecto de los atributos naturales de valores únicos (ej.: número de legajo) en las clases:
 - o Si la clase tiene uno de estos atributos, en el **DER** será una PK (clave natural).
 - o Si la clase NO tiene uno de estos atributos, en el **DER** se deberá crear una columna **id** para que sea PK (clave subrogada, autogenerada).

Del DER (diagrama entidad-relación) al DC (diagrama de clases):

- Si en el **DER** hay una tabla intermedia (es decir, una tabla creada para solucionar la relación many-to-many) donde se guardan...
 - o ... solamente FKs → en el **DC** no hay que crear un objeto intermedio.
 - o ... algo más que FKs → en el **DC** sí hay que crear un objeto intermedio.
- Las claves subrogadas (identificadores únicos -numéricos- que no se desprenden de los datos de cada registro) del **DER** deberían estar en el **DC**.

¿Qué tipos de datos usar para los atributos (Diagrama de Clases) y campos (DER)?

	Diagrama de Clases	DER
Nombres. Apellidos. Nombres de Usuario. Contraseñas. CUITs. Emails. Tipos de Documento. Descripciones. Opiniones Libres. Nombres de una Calle.	String	VARCHAR(n) NVARCHAR(n)
IDs. Cantidades de ... Teléfonos. Números de Documento. Alturas de una Calle. Números de Piso (Depto.).	Integer	INTEGER(n)
Precios. Sueldos. Distancias (Alturas, Longitudes, Radios).	Double	DECIMAL(t,d) t: dígitos totales d: dígitos en mantisa
Fechas Horarios. Instantes (Fecha-Hora).	LocalDate LocalTime LocalDateTime	DATE TIME DATETIME
Valores booleanos.	Boolean	BIT